# (Optional) Extra Practice Problem

---

## A. The Homework Bot (Easy)

### Problem statement

The lab built a homework bot. It does not give long speeches. It only answers one question: did this student pass the AI quiz?
Write a function check_pass(marks) that **returns** True if marks >= 40, otherwise False. Do not print inside the function. Add a short docstring — even bots deserve documentation.
Then read one quiz mark, call the function, and print Pass or Fail.

### Input

One line: an integer marks (−100 ≤ marks ≤ 100).

### Output

Print Pass or Fail.

### Examples

**Input**
```
73
```
**Output**
```
Pass
```
**Input**
```
39
```
**Output**
```
Fail
```

---

## B. Three Training Runs (Easy)

### Problem statement

You trained a tiny model three times. Same data, three different random seeds. The accuracies came out different — models have moods, like you before coffee.
Write average(a, b, c) that **returns** the mean accuracy as a float: (a + b + c) / 3. Call it with **positional** arguments and print only the returned value. The report needs a decimal, so the supervisor believes you actually divided.

## Input

One line: three numbers a b c (the three accuracies).

## Output

Print the average as a float (for example 20.0, not 20).

## Example

**Input**
```
10 20 30
```
**Output**
```
20.0
```

---

# C. New Interns at the AI Lab (Easy)

## Problem statement

Two new interns just walked in. Security will not open the GPU room without an ID card. Your job is the card printer.
Write student_card(name, roll, dept) that prints:
`<name> | Roll: <roll> | Dept: <dept>`
You will get two interns. Call the function **twice** with **positional** arguments: name, then roll, then dept. Swap the order and someone from EEE will get into the CSE GPU queue. Chaos.

## Input

Two lines. Each line: name roll dept (name and dept are one word).

## Output

Two ID lines, same order as input.

## Example

**Input**
```
Rafi 101 CSE
Mim 102 EEE
```
**Output**
```
Rafi | Roll: 101 | Dept: CSE
Mim | Roll: 102 | Dept: EEE
```

# D. The Model Registry (Medium)

## Problem statement

The lab keeps a registry of models. If nobody mentions a learning rate, the senior's rule is: **use 0.01**. "If you forget lr, the default will remember for you."
Write model_info(name, layers, lr=0.01) that **returns**:
`Model: <name>, Layers: <layers>, LR: <lr>`
Today there are t registration requests.

- Type 1 — intern is lazy: only name and layers. Use the default lr. Call with positional arguments.
- Type 2 — intern typed fields in a weird order. Call with **keyword** arguments (layers=..., name=...).
- Type 3 — intern is brave and sets a custom learning rate.

Print the registry line for each request.

## Input

First line: integer t (1 ≤ t ≤ 10).
Each of the next t lines:
1 name layers  or  2 name layers  or  3 name layers lr

## Output

t registry lines.

## Example

**Input**

```
3
1 TinyNet 3
2 ResNet 5
3 FastNet 2 0.001
```

**Output**

```
Model: TinyNet, Layers: 3, LR: 0.01
Model: ResNet, Layers: 5, LR: 0.01
Model: FastNet, Layers: 2, LR: 0.001
```

# E. Clean Images, Then Features (Medium)

## Problem statement

You have a folder of face-image IDs for an age-detection toy model. Odd IDs are blurry selfies. Even IDs are clean.
Keep only the clean (even) IDs, then turn each one into a dummy feature by squaring it. Yes, squaring an ID is a silly feature. The senior said: “First learn filter and map. Fancy embeddings can wait.”
You must use filter() + lambda to keep evens, then map() + lambda to square. Print the final list of features.

## Input

First line: integer n (1 ≤ n ≤ 100) — how many IDs.
Second line: n integers — the image IDs.

## Output

Print a Python list of squared even IDs, same order. Example: [4, 16, 36].

## Example

**Input**

```
6
1 2 3 4 5 6
```

**Output**

```
[4, 16, 36]
```

**Hint:** আগে ব্লারি ফেস ফেলে দাও (filter), পরে ফিচার বানাও (map)। উল্টো করলে 1-এর স্কয়ারও ঢুকে পড়বে — ডেটাসেট নষ্ট।

---

# F. GPU Can Eat Only Three Images (Hard)

## Problem statement

TinyNet is hungry. You have a list of training samples (think: face images for age-group classification). The lab GPU is small. It can swallow only batch_size samples at a time.
Write a **generator** batch_generator(data, batch_size) that yields one batch (a list) at a time — like giving the GPU one plate, not the whole buffet. The last plate may have fewer samples. If batch_size < 1, the GPU is off: yield nothing.
Also write batch_mean(batch) that **returns** the average of the numbers in that batch. The senior uses this as a tiny sanity check before real training.
Read the samples and the batch size. For each batch print:

```
[1, 2, 3] mean=2.0
```

## Input

First line: two integers n and batch_size (n ≥ 1).
Second line: n integers — the sample values.

## Output

One line per batch: the batch list, a space, then mean= and the mean.
If batch_size < 1, print nothing.

## Example

**Input**

```
8 3
1 2 3 4 5 6 7 8
```

**Output**

```
[1, 2, 3] mean=2.0
[4, 5, 6] mean=5.0
[7, 8] mean=7.5
```